Part 2: LoRA Fine-tuning a Small LLM for Tool Calling

EN4554 Deep Learning for Vision · Assignment 1 Praveen V.V.J. (220491B) · Sulojan R. (220626V)

1. Setup

- **Model:** Qwen2.5-0.5B-Instruct, used through its built-in tool-calling chat template (calls are emitted as `<tool_call>{json}</tool_call>`). Greedy decoding, at most 128 new tokens per turn.
- **Tools (4):** `calculator` (arithmetic expression), `unit_convert` (length, mass, °C/°F), `get_weather` (made-up weather for 6 Sri Lankan cities) and `lookup_order` (60 made-up orders, ORD-1000 to ORD-1059). The made-up values mean the answers cannot come from the model's memory.
- **Agent loop:** generate → parse any tool call → execute it → append the result as a tool message → repeat. The loop stops on a plain-text reply or after 4 steps.
- **Data:** template-generated, with gold answers computed by actually running the tools. **Train: 1,200** requests (856 single-call, 118 two-call chains, 226 needing no tool). **Test: 100** requests (68 single-call, 5 chains, 27 no-tool), built from a separate set of templates with different wording and sentence structure. No test request appears in the training set (checked in code).
- **LoRA:** r = 16, α = 32, dropout 0.05 on all attention and MLP projections (q, k, v, o, gate, up, down), giving 8.80 M trainable parameters (1.75 %). The loss is computed on assistant tokens only; system, user and tool-result tokens are masked. 2 epochs, learning rate 2e-4 (cosine, 5 warm-up steps), effective batch 16 (micro-batch 2 × 8 accumulation, because Qwen's 152 k-token vocabulary ran out of memory at batch 8), fp16 mixed precision, seed 42. Training took 150 steps and 6 min 42 s on a Colab T4, and the loss fell from 0.34 to 0.0003.

2. Baseline vs. LoRA (same 100-request test set)

Metric	Definition (denominator)	Baseline	LoRA	Change
Call validity	made ≥ 1 call, all well-formed (tool requests, n = 73)	83.6 %	**100.0 %**	+16.4
Call accuracy	right tool(s) and right arguments (n = 73)	68.5 %	**90.4 %**	+21.9
Abstention	answered without a tool when none was needed (n = 27)	59.3 %	**96.3 %**	+37.0
Task success	final answer contains the gold facts, end to end (n = 100)	55.0 %	**92.0 %**	+37.0

The baseline never produced malformed JSON. All of its validity failures were tool requests where it made no call at all.

3. Failure analysis

Category (one per request)	Base	LoRA
Ignored tool result (right call, wrong final answer)	15	0
Missed call (no call, or chain stopped early)	14	0
Unnecessary call (no-tool request)	11	1
Wrong answer to a no-tool request	7	0
Wrong tool	6	4
Wrong arguments	3	3
Wrong format	0	0
Total failures	**56**	**8**

Pairing the same test requests: LoRA **fixed 49** of the 56 baseline failures, **left 7 unfixed** and **introduced 1**. All 8 LoRA failures were inspected (only 8 exist, fewer than the 10 the brief asks for), along with 10 of the baseline failures.

Baseline errors were decision errors. It asked the user for data instead of calling ("should I pack a coat?", then "could you provide the current temperature?"). It called tools for general knowledge (`lookup_order(order_id="RAM")` for "What does RAM stand for?"). It refused easy questions ("I can only provide details about mathematical operations and weather"). It also misreported correct results ("the backpack is expected to be back in stock in 6 days"). **LoRA fixed** every ignored result, every refusal, every wrong-tool error, 10 of the 11 unnecessary calls and 10 of the 14 missed calls.

Not fixed (7): "Take *b* away from *a*" has wrong arguments in both models, and LoRA computes *b* − *a* (2 cases). The boiling-point question still triggers a call, now `calculator("0*100")`, answered as "0 °C" (1). "I'm travelling to *city*, should I pack a coat?" went from a missed call to the wrong tool, `calculator("1*24")`, answered as "24 hours until you arrive" (4). **Introduced (1):** "Vavuniya weather report, please" now calls `get_weather(city="Lisbon")`.

The remaining LoRA errors share one cause: **over-fitting to the training templates** (loss ≈ 2e-4 by epoch 2). The model keys on surface patterns, such as reading "travelling to *city*" as the trained "how many hours until … arrives" chain, and operand order in subtraction. It is not reading the intent. More varied phrasings, paraphrase augmentation, or a single epoch would be the next step. The ignored-result count is also somewhat inflated by a strict answer check, which requires both the order status and the item.

4. Would a better prompt alone have been enough?

Only partly: the baseline already produced well-formed calls through Qwen's built-in tool template, so its failures were decisions (whether to call a tool at all in 32 cases, how to report the result in 15), and a stricter system prompt with a few worked examples could plausibly reduce these without any training. However, a 0.5 B model follows long instructions unreliably, and LoRA removed 49 of the 56 failures after under seven minutes of training (we did not run a prompt-only ablation to measure the gap). The errors LoRA left come from template over-fitting, which better training data, not a better prompt, would address.